Use Cases

for

SnapCode

Version 1.0 approved

Prepared by Hardik Prajapati (24CSB1A19), Hisham (24CSB1A20), Guneev (24CSB1A18)

National Institute of Technology Warangal

31/Aug/2026

Revision History

Name	Date	Reason For Changes	Version
Hardik, Hisham, Guneev	31/Aug/2026	Initial release of Use Case Document for SnapCode new features	1.0

Use Cases

UC-1.0: Generate Flutter Source Code from Visual Blocks

Use Case ID:	UC-1.0		
Use Case Name:	Generate Flutter Source Code from Visual Blocks		
Created By:	Hardik, Hisham, Guneev	Last Updated By:	Hardik, Hisham, Guneev
Date Created:	31/Aug/2026	Date Last Updated:	31/Aug/2026
Actors:	Student / Novice Programmer, CS Instructor, Prototyper		
Description:	The system parses the visual block layout (AST) and translates it into clean, structured Flutter/Dart code in real-time or on-demand.		
Preconditions:	1. The user has assembled a valid set of logic blocks on the canvas workspace.		
Postconditions:	1. The system outputs valid, readable Flutter/Dart code corresponding to the visual blocks.		
Normal Flow:	1. User completes the assembly of visual blocks. 2. User initiates the code generation process (or system detects change). 3. System parses the visual block AST. 4. System translates the AST into Flutter/Dart syntax. 5. System displays the generated code in the side-by-side preview panel.		
Alternative Flows:	UC-1.0.AC.1: Real-time generation: The system continuously generates and updates the Flutter code as the user drags and drops blocks.		
Exceptions:	UC-1.0.EX.1: Unconnected blocks: If blocks are not properly snapped, the system ignores them or highlights an error.		
Includes:	None		
Priority:	High		
Frequency of Use:	Very frequent (continuous during coding)		
Business Rules:	Generated code must comply with Dart language specifications and Flutter idiomatic structure guidelines.		
Special Requirements:	Block-to-code compilation cycle must be under 200ms.		
Assumptions:	The visual blocks map deterministically to valid syntax constructs in Dart/Flutter.		
Notes and Issues:	None.		

UC-2.0: Live Cross-Platform Application Preview

Use Case ID:	UC-2.0		
Use Case Name:	Live Cross-Platform Application Preview		
Created By:	Hardik, Hisham, Guneev	Last Updated By:	Hardik, Hisham, Guneev
Date Created:	31/Aug/2026	Date Last Updated:	31/Aug/2026
Actors:	Student / Novice Programmer, Prototyper		
Description:	The user views a live, executing preview of the application logic they have built using blocks, rendered for the target platform (Web, Mobile, Desktop).		
Preconditions:	1. Valid Flutter code has been generated from the block canvas.		
Postconditions:	1. A live app preview is rendered in the integrated dashboard.		
Normal Flow:	1. User clicks the "Preview" button on the central dashboard. 2. System compiles the generated Flutter code for the selected platform (e.g., Web). 3. System renders the interactive application preview in a side panel or modal. 4. User interacts with the previewed application.		
Alternative Flows:	None.		
Exceptions:	UC-2.0.EX.1: Compilation failure: If the generated code fails to compile, the system displays an error message detailing the issue.		
Includes:	Includes UC-1.0 (Generate Flutter Source Code)		
Priority:	High		
Frequency of Use:	Frequent (during testing and debugging)		
Business Rules:	Preview must reflect the current state of the block canvas accurately.		
Special Requirements:	Requires modern browser or desktop runtime support for live execution.		
Assumptions:	The generated Flutter code relies on stable Flutter SDK channels.		
Notes and Issues:	None.		

UC-3.0: Export Production-Ready Flutter Project

Use Case ID:	UC-3.0		
Use Case Name:	Export Production-Ready Flutter Project		
Created By:	Hardik, Hisham, Guneev	Last Updated By:	Hardik, Hisham, Guneev
Date Created:	31/Aug/2026	Date Last Updated:	31/Aug/2026
Actors:	Student / Novice Programmer, Prototyper		
Description:	The user exports their visual block project as a standard, independent Flutter codebase that can be compiled and distributed outside of SnapCode.		
Preconditions:	1. The user has a saved project with generated Flutter code.		
Postconditions:	1. A complete Flutter project (e.g., ZIP file containing pubspec.yaml, lib folder, etc.) is downloaded to the user's local machine.		
Normal Flow:	1. User selects "Export Project" from the dashboard menu. 2. System packages the generated Dart code, project assets, and necessary Flutter configuration files. 3. System compresses the project into a ZIP archive. 4. System prompts the user to download the archive.		
Alternative Flows:	UC-3.0.AC.1: Export to GitHub: User chooses to export directly to a linked GitHub repository instead of downloading a ZIP.		
Exceptions:	UC-3.0.EX.1: Storage quota exceeded: If exporting to cloud/repository and storage is full, notify the user.		
Includes:	Includes UC-1.0		
Priority:	High		
Frequency of Use:	Occasional (when ready for deployment or manual code editing)		
Business Rules:	Exported code must not contain proprietary SnapCode lock-in mechanisms; it must be standard Flutter code.		
Special Requirements:	Standardized project structure generation.		
Assumptions:	User has basic knowledge of how to run a Flutter project if they export it.		
Notes and Issues:	None.		

UC-4.0: Interactive Magnetic Block Assembly with Syntax Constraints

Use Case ID:	UC-4.0		
Use Case Name:	Interactive Magnetic Block Assembly with Syntax Constraints		
Created By:	Hardik, Hisham, Guneev	Last Updated By:	Hardik, Hisham, Guneev
Date Created:	31/Aug/2026	Date Last Updated:	31/Aug/2026
Actors:	Student / Novice Programmer, CS Instructor		
Description:	Users build application logic by dragging blocks onto the canvas. The system enforces syntax correctness by only allowing blocks to magnetically snap together if they form valid logic.		
Preconditions:	1. User has the workspace canvas open.		
Postconditions:	1. The canvas contains a tree of syntactically valid logic blocks.		
Normal Flow:	1. User selects a block from the block palette. 2. User drags the block onto the canvas. 3. User drags the block near another block's connector. 4. System highlights valid connection points. 5. User releases the mouse; blocks magnetically snap together.		
Alternative Flows:	None.		
Exceptions:	UC-4.0.EX.1: Invalid connection attempt: If the user tries to connect incompatible blocks, the system rejects the connection and the block returns to its previous position or floats freely.		
Includes:	None		
Priority:	Critical		
Frequency of Use:	Very Frequent (core interaction)		
Business Rules:	Magnetic constraints must perfectly mirror Dart/Flutter type safety and syntax rules.		
Special Requirements:	Low latency drag responsiveness (60 FPS rendering). Zero crash rate during block snapping.		
Assumptions:	None.		
Notes and Issues:	None.		

UC-5.0: Cloud-Based Project State Management

Use Case ID:	UC-5.0		
Use Case Name:	Cloud-Based Project State Management		
Created By:	Hardik, Hisham, Guneev	Last Updated By:	Hardik, Hisham, Guneev
Date Created:	31/Aug/2026	Date Last Updated:	31/Aug/2026
Actors:	Student / Novice Programmer, CS Instructor		
Description:	The system saves the visual workspace state, block AST, and user configurations to a backend database, allowing users to load and resume their work from any device.		
Preconditions:	1. User is logged into SnapCode.		
Postconditions:	1. The project state is persistently stored in the database.		
Normal Flow:	1. User makes changes to the block canvas. 2. User clicks "Save" (or auto-save triggers). 3. System serializes the block tree and workspace configuration. 4. System transmits data to the backend API. 5. Backend updates the database with the latest project state. 6. System displays a "Save Successful" notification.		
Alternative Flows:	UC-5.0.AC.1: Load Project: User selects a project from their dashboard, and the system deserializes the data to rebuild the block canvas.		
Exceptions:	UC-5.0.EX.1: Network disconnection: If the network drops during save, the system caches changes locally and retries when reconnected.		
Includes:	None		
Priority:	Important		
Frequency of Use:	Frequent		
Business Rules:	Project data must be tied securely to the user's account.		
Special Requirements:	Backend API and Database must support JSON serialization of AST.		
Assumptions:	Application is accessible online.		
Notes and Issues:	None.		

UC-6.0: Access Contextual Flutter/Dart Syntax Tooltips

Use Case ID:	UC-6.0		
Use Case Name:	Access Contextual Flutter/Dart Syntax Tooltips		
Created By:	Hardik, Hisham, Guneev	Last Updated By:	Hardik, Hisham, Guneev
Date Created:	31/Aug/2026	Date Last Updated:	31/Aug/2026
Actors:	Student / Novice Programmer		
Description:	To aid learning, the system provides embedded contextual tooltips that explain how a specific visual block maps to Flutter/Dart syntax when the user interacts with it.		
Preconditions:	1. The block canvas is populated with blocks.		
Postconditions:	1. User gains understanding of the underlying code for a block.		
Normal Flow:	1. User hovers the mouse over a specific logic block on the canvas. 2. System displays a tooltip containing an explanation of the block's function. 3. System displays the exact Dart code snippet that this specific block generates.		
Alternative Flows:	None.		
Exceptions:	None.		
Includes:	None		
Priority:	Medium		
Frequency of Use:	Occasional (as needed for learning)		
Business Rules:	Tooltips must accurately reflect the generated code.		
Special Requirements:	Embedded contextual tooltips must not obscure the workspace unnecessarily.		
Assumptions:	Target audience (novices) uses this to bridge the gap between visual and text coding.		
Notes and Issues:	None.		